1. ปัญหา

ระบบนี้ใช้ตรวจหาบัญชีที่น่าสงสัยจากไฟล์ log ของการเข้าสู่ระบบของพนักงาน โดยไฟล์เป็นรูปแบบ CSV หนึ่งการเข้าสู่ระบบหนึ่งครั้ง ประกอบด้วยเวลา ชื่อผู้ใช้ ที่อยู่ IP และสถานะ ซึ่งมีสองค่าคือ FAILED (เข้าสู่ระบบไม่สำเร็จ) และ SUCCESS (เข้าสู่ระบบสำเร็จ) เป้าหมายคือหาบัญชีที่เข้าสู่ระบบผิดซ้ำในเวลาสั้นๆ ซึ่งอาจเป็นการเดารหัสผ่าน โดยกำหนดเงื่อนไขไว้ดังนี้

บัญชีที่ผิดปกติ: คือบัญชีที่มี FAILED ติดต่อกันตั้งแต่ 3 ครั้งขึ้นไป โดยครั้งแรกกับครั้งสุดท้ายห่างกันไม่เกิน 5 นาที ถ้าห่างกัน 5 นาทีพอดีนับว่าผิดปกติ

- **SUCCESS :** ถ้าระหว่างที่ FAILED มี SUCCESS แทรกเข้ามา ให้เริ่มนับใหม่ เช่น FAILED 2 ครั้ง ตามด้วย SUCCESS แล้ว FAILED อีก 2 ครั้ง จะไม่ถูกแจ้งเตือน เพราะไม่มีชุดไหนครบ 3 ครั้ง
- **ต้องนับแยกรายบัญชี:** เพราะ log ของหลายบัญชีปนกันอยู่ในไฟล์เดียว และบรรทัดอาจไม่เรียงตามเวลา จึงต้องจัดกลุ่มตามชื่อผู้ใช้และเรียงตามเวลาก่อนนับ
- **Input:** ไฟล์ log
- **Output:** รายชื่อบัญชีที่ผิดปกติ พร้อมจำนวนครั้งและช่วงเวลาที่พบ เพื่อให้ผู้ดูแลระบบตรวจสอบต่อ

2. การออกแบบอัลกอริทึม

2.1 เตรียมข้อมูล (ใช้ร่วมกันทั้ง A และ B)

ฟังก์ชัน load_logs อ่านไฟล์ แปลงเวลาเป็น datetime จัดกลุ่มตาม username และเรียงเหตุการณ์ของแต่ละบัญชีตามเวลา หากพบบรรทัดที่ผิดรูปแบบ (รูปแบบวันเวลาไม่ถูกต้อง ช่องชื่อผู้ใช้ว่าง หรือสถานะเป็นค่าอื่นนอกจาก FAILED และ SUCCESS) ระบบจะหยุดพร้อมแจ้งเลขบรรทัด เพื่อไม่ให้ข้อมูลเสียทำให้ผลตรวจผิด ขั้นตอนนี้มีประสิทธิภาพด้านเวลาเป็น O(n log n)

2.2 อัลกอริทึม A: Sliding Window แบบ Two-Pointer (detect_a)

แยกเหตุการณ์ของแต่ละบัญชีเป็นชุด fail ต่อเนื่อง แล้วสแกนแต่ละชุดด้วยสองตัวชี้ทั้งซ้ายและขวาเดินหน้าอย่างเดียว ไม่ย้อนกลับ จึงสแกนแต่ละชุดด้วยเวลา O(n)

การแจ้งเตือนเฉพาะหน้าต่างที่ขยายขวาไม่ได้แล้ว ทำให้แต่ละช่วงถูกรายงานเป็นช่วงสูงสุด ไม่ซ้ำเป็นช่วงย่อยหลายอัน

2.3 อัลกอริทึม B: Brute Force (detect_b)

ใช้เปรียบเทียบ โดยแยกชุด FAILED เหมือน A แต่ให้ทุกเหตุการณ์ในชุดเป็นจุดเริ่มต้น แล้วขยายไปทางขวาทีละตัว ถ้ายังห่างจากจุดเริ่มไม่เกิน 5 นาที แล้วต่อจากนั้นแจ้งเตือนเมื่อนับได้ตั้งแต่ 3 หน้าต่างนั้นขยายไปทางซ้ายไม่ได้แล้ว ลูปซ้อนนี้ทำให้ต้องไล่เหตุการณ์ซ้ำจากทุกจุดเริ่ม

2.4 เปรียบเทียบความซับซ้อน

เกณฑ์การเปรียบเทียบ	**อัลกอริทึม A: Sliding Window**	**อัลกอริทึม B: Brute Force**
วิธีคิด	ใช้ตัวชี้ left/right เดินหน้าอย่างเดียว	ลองทุกจุดเริ่มต้น แล้วขยายไปทางขวาจนหลุด 5 นาที
เวลา (สแกน)	O(n)	$O(n^2)$ กรณีแย่สุด (มี FAILED จำนวนมากในช่วง 5 นาที
เวลา (รวมการเรียง)	O(n log n)	$O(n^2)$ กรณีแย่สุด
พื้นที่	O(n)	O(n)

3. เหตุผลในการเลือกอัลกอริทึม

- **ผลลัพธ์ตรงกับอัลกอริทึม B ที่ตรวจทุกกรณี:** A และ B ให้ผลลัพธ์ตรงกันในไฟล์ตัวอย่างและกรณีทดสอบขอบทั้ง 5 กรณี จึงใช้ B เป็น baseline ตรวจความถูกต้องของ A ได้

- **รับ log จำนวนมากได้ดีกว่า:** ทั้งสองวิธีต้องเรียงข้อมูลเหมือนกัน (O(n log n)) แต่ขั้นสแกนของ A เป็น O(n) เสมอ ส่วน B เป็น $O(n^2)$ เมื่อมี FAILED จำนวนมากอยู่ในช่วง 5 นาที ซึ่งเป็นลักษณะของการโจมตีแบบเดารหัสผ่านพอดีเป็นสถานการณ์ที่ระบบต้องทำงานได้ดีที่สุด
- **เหมาะกับลักษณะของโจทย์**: เงื่อนไข "ช่วงเวลา 5 นาที" คือหน้าต่างที่เลื่อนไปตามเวลา และข้อมูลเรียงตามเวลาแล้ว จึงเหมาะกับ sliding window โดยตรง
- **ไม่เสียพื้นที่เพิ่ม:** ทั้งสองวิธีใช้พื้นที่ O(n) เท่ากัน (ต้องเก็บ log และสำเนาที่เรียงแล้ว) จึงไม่มีข้อแลกเปลี่ยนด้านหน่วยความจำ
- **ข้อเปรียบเทียบเพิ่มเติม:** B ยังมีข้อดีคือเขียนและตรวจด้วยตาได้ง่าย จึงเหมาะเป็นตัวเทียบ แต่ไม่เหมาะเป็นตัวใช้งานจริงกับ log ขนาดใหญ่ ส่วนวิธีแบ่งช่องเวลาคงที่ (fixed time-bucket) ถูกตัดออกเพราะอาจพลาดการโจมตีที่คร่อมขอบช่อง เช่น 2 ครั้งท้ายช่องแรกกับ 1 ครั้งต้นช่องถัดไป ทั้งที่รวมกันอยู่ใน 5 นาที